

9. TRAVELLING SALESMAN PROBLEM

Aim:

To implement the Travelling Salesman Problem (TSP) using the Brute Force Search algorithm in Python and determine the minimum-cost tour that visits every city exactly once and returns to the starting city.

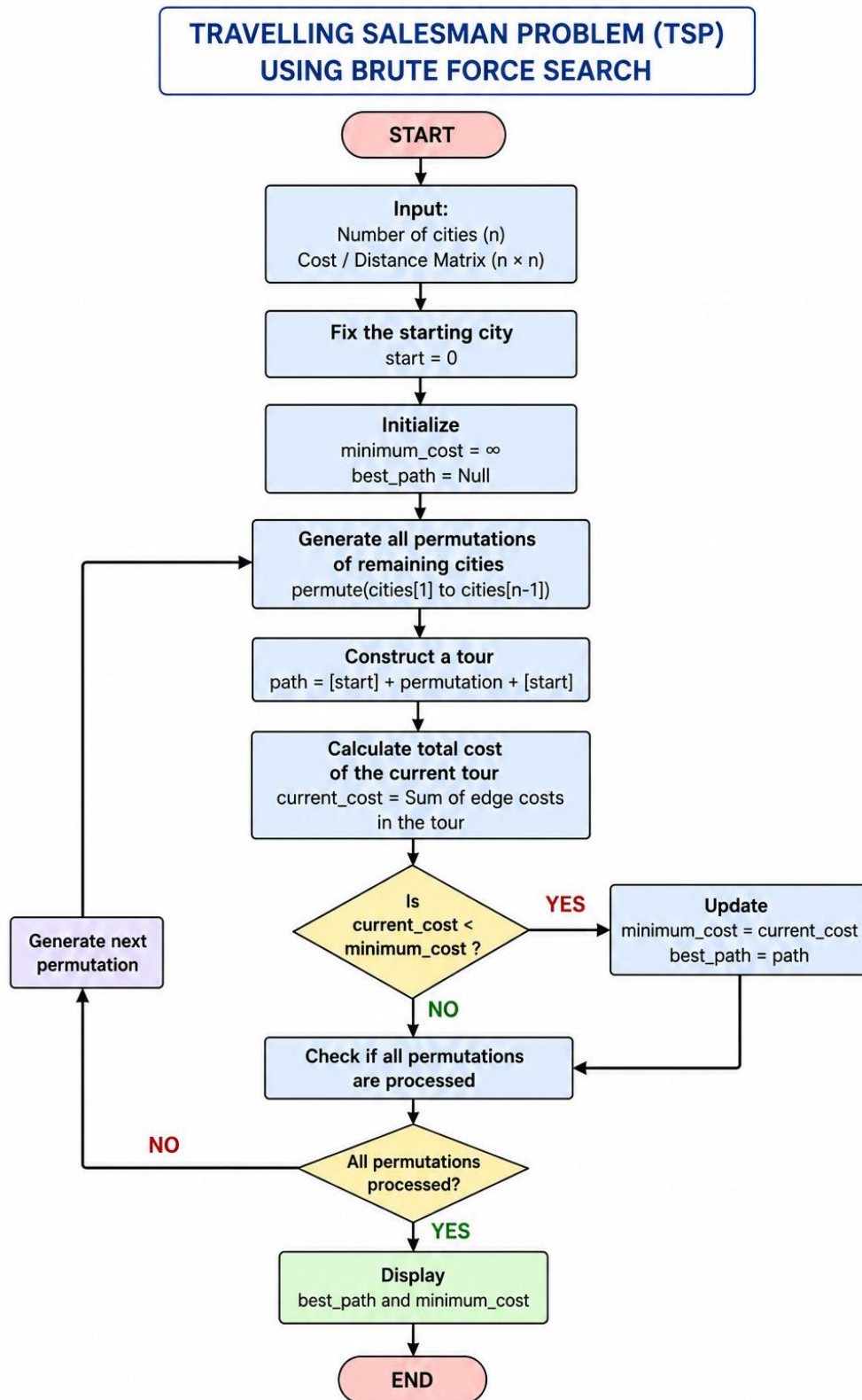
Objective :

To find the shortest possible route that visits all cities exactly one time and returns to the starting city by evaluating all possible tours.

Algorithm:

1. Represent the cities using a cost (distance) matrix.
2. Select the first city as the starting point.
3. Generate all possible permutations of the remaining cities.
4. Calculate the total travel cost for each tour.
5. Compare the cost with the current minimum cost.
6. If the current tour has a lower cost, update the minimum cost and best path.
7. Continue until all possible tours are evaluated.
8. Display the optimal path and minimum travel cost.

Flowchart :



Code:

```
from itertools import permutations

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

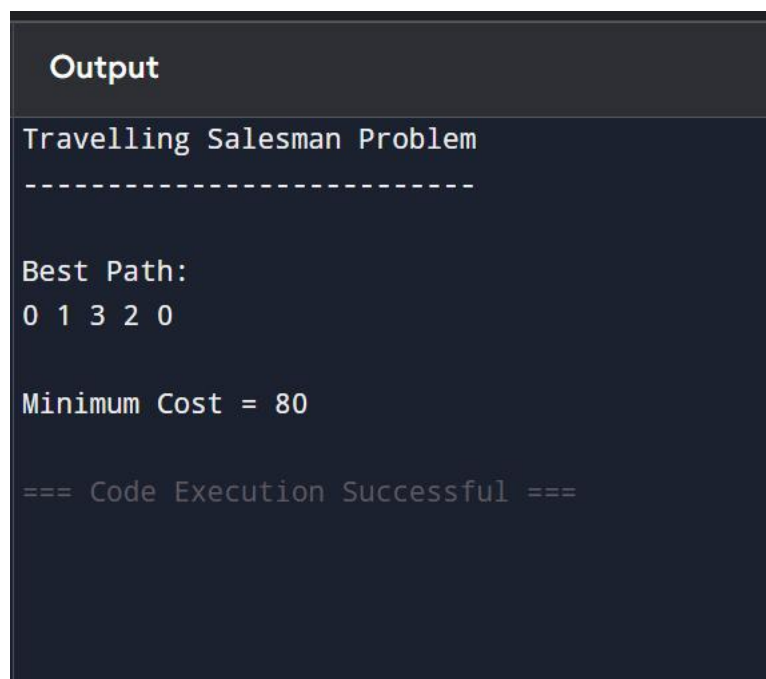
n = len(graph)
cities = list(range(n))
start = 0
minimum_cost = float('inf')
best_path = None

for path in permutations(cities[1:]):
    current_cost = 0
    current_city = start
    for city in path:
        current_cost += graph[current_city][city]
        current_city = city
    current_cost += graph[current_city][start]
    if current_cost < minimum_cost:
        minimum_cost = current_cost
        best_path = (start,) + path + (start,)

print("Travelling Salesman Problem")
```

```
print("-----")
print("\nBest Path:")
for city in best_path:
    print(city, end=" ")
print("\n\nMinimum Cost =", minimum_cost)
```

Output:



```
Output
Travelling Salesman Problem
-----

Best Path:
0 1 3 2 0

Minimum Cost = 80

=== Code Execution Successful ===
```

Result :

The Travelling Salesman Problem was successfully solved using the Brute Force Search algorithm. The algorithm examined all possible tours and identified the route with the minimum travel cost.